

Laboratorio di Calcolo

A.A. 2026/2027, Canale A-Dec

Giacomo Artoni, Lorenzo Caprini

Numeri pseudocasuali

Il caso in fisica

Molti fenomeni fisici sono **intrinsecamente casuali**, o così complicati da poter essere descritti solo **statisticamente**.

- **Decadimento radioattivo:** non si sa *quando* un nucleo decadrà, solo con quale **probabilità** per unità di tempo.
- **Moto browniano:** un granello di polline urtato da miliardi di molecole d'acqua.
- **Errori di misura:** le fluttuazioni casuali viste nella lezione 18.
- **Rivelatori di particelle:** quale traccia lascia una particella, quanta energia deposita…

- Per **simulare** questi fenomeni al computer servono numeri **casuali**.
- Esempi in questa lezione: cammino casuale e decadimento radioattivo.
- Più avanti: calcolare integrali con numeri casuali, il metodo **Monte Carlo**.

Ma un computer esegue istruzioni **deterministiche**: come può produrre numeri casuali?

Casuale o pseudocasuale?

- Un programma, a parità di input, fa **sempre** la stessa cosa: non può "tirare a sorte".
- Si usa un **algoritmo** che produce una sequenza di numeri che **sembra** casuale:

$$x_{n+1} = f(x_n)$$

- Il primo numero x_0 si chiama **seme** (*seed*).
- Stesso seme → stessa sequenza: i numeri sono **pseudocasuali**.

Cosa chiediamo a un buon generatore

- **Uniformità:** tutti i valori compaiono con la stessa frequenza.
- **Indipendenza:** un numero non dà informazioni sul successivo.
- **Periodo lungo:** prima o poi la sequenza si ripete (i valori possibili sono finiti!), ma deve succedere il più tardi possibile.
- **Velocità:** nelle simulazioni se ne usano miliardi.

Che sia deterministico è anche un **vantaggio**: una simulazione si può **ripetere** identica, per cercare un errore.

Il generatore lineare congruenziale

Il più semplice e storicamente più usato (D. H. Lehmer, 1949):

$$x_{n+1} = (a\,x_n + c) \bmod m$$

- a (moltiplicatore), c (incremento) e m (modulo) sono **interi** fissati.
- Il resto della divisione per m (lezione 11) dà sempre un valore tra 0 e $m-1$.
- Al massimo m valori diversi: il **periodo** è al più m.
- Dividendo per m si ottiene un numero in $[0, 1)$: $u_n = x_n / m$.
- Nei generatori veri m è enorme, ad esempio 2^{31} o 2^{64}.

Esempio: $m = 16, a = 5, c = 3, x_0 = 7$

n	x_n	$ax_n + c$	x_{n+1}
0	7	38	6
1	6	33	1
2	1	8	8
3	8	43	11
4	11	58	10

Il generatore in C e il suo periodo

```c
#include <stdio.h>

int main() {
    const int m = 16, a = 5, c = 3;
    int x = 7;                          // seme
    int i;

    for (i = 0; i < 20; i++) {
        printf("%d ", x);
        x = (a * x + c) % m;
    }
    printf("\n");
    return 0;
}
```

```
$ ./lcg
7 6 1 8 11 10 5 12 15 14 9 0 3 2 13 4 7 6 1 8
```

- Compaiono **tutti** i 16 valori da 0 a 15, poi la sequenza **si ripete**: periodo 16, il massimo possibile.
- Con `a = 3, c = 0` invece:

```
7 5 15 13 7 5 15 13 7 5 15 13 7 5 15 13 7 5 15 13
```

- Periodo 4: la scelta di a, c, m è **cruciale**, e va lasciata agli esperti.
- Guardate la prima sequenza: dispari, pari, dispari, pari… Le cifre binarie **basse** di un LCG sono poco casuali!

La funzione rand

La libreria standard (`#include <stdlib.h>`) fornisce un generatore già pronto.

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int i;

    printf("RAND_MAX = %d\n", RAND_MAX);
    for (i = 0; i < 4; i++) {
        printf("%d\n", rand());
    }
    return 0;
}
```

- `rand()` restituisce un `int` tra `0` e `RAND_MAX` (estremi inclusi).
- `RAND_MAX` è una costante definita in `stdlib.h` : vale **almeno** 32767.

```
$ ./primi_rand
RAND_MAX = 2147483647
16807
282475249
1622650073
984943658
$ ./primi_rand
RAND_MAX = 2147483647
16807
282475249
1622650073
984943658
```

- Due esecuzioni: **gli stessi numeri**!
- Output ottenuto su un Mac: l'algoritmo dipende dalla libreria, e in laboratorio (Linux) otterrete numeri **diversi**.

Il seme: srand

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    unsigned int seme;
    int i;

    printf("Seme: ");
    scanf("%u", &seme);
    srand(seme);
    for (i = 0; i < 3; i++) {
        printf("%d\n", rand());
    }
    return 0;
}
```

```
$ ./seme
Seme: 42
705894
1126542223
1579310009
$ ./seme
Seme: 42
705894
1126542223
1579310009
$ ./seme
Seme: 43
722701
1409017472
1054476435
```

- `srand(seme)` inizializza il generatore; il seme è un `unsigned int`.
- Senza `srand`, è come se si chiamasse `srand(1)`.

- Stesso seme → **stessa sequenza**; seme diverso → sequenza diversa.
- Anche qui i valori esatti cambiano da sistema a sistema.

Seme fisso o seme dal tempo?

```c
#include <stdlib.h>
#include <time.h>
...
unsigned int seme = time(NULL);
srand(seme);
printf("seme = %u\n", seme);
```

```
$ ./tempo
seme = 1792051498
...
$ ./tempo
seme = 1792051499
...
```

- `time(NULL)` (da `time.h`) dà i **secondi** trascorsi dal 1° gennaio 1970: cambia a ogni esecuzione…
- …ma solo se passa almeno **un secondo**!

Seme fisso	Seme da `time(NULL)`
Risultati **riproducibili**	Ogni esecuzione è diversa
Ideale per il **debug** e i test	Per raccogliere statistica da più esecuzioni
Si può confrontare con i compagni (stessa macchina)	Va **stampato**, per poter ripetere l'esecuzione

`srand` si chiama **una sola volta**, all'inizio del programma. Chiamarla nel ciclo con `time(NULL)` reimposta sempre lo stesso seme: si ottengono numeri **tutti uguali**!

Numeri reali uniformi

$$u = \frac{\texttt{rand()}}{\texttt{RAND_MAX} + 1.0} \in [0, 1) \qquad x = a + (b - a)\, u \in [a, b)$$

```c
const double pi = 3.141592653589793;
const double a = 0.0, b = 2.0 * pi;          // [rad]

srand(2026);
for (i = 0; i < 4; i++) {
    double u   = rand() / (RAND_MAX + 1.0);  // in [0, 1)
    double phi = a + (b - a) * u;            // in [a, b)
    printf("u = %f   phi = %f rad\n", u, phi);
}
```

```
u = 0.015856   phi = 0.099628 rad
u = 0.495559   phi = 3.113688 rad
u = 0.856383   phi = 5.380812 rad
u = 0.225167   phi = 1.414768 rad
```

- Esempio: direzione φ di una particella emessa a caso nel piano.
- `+ 1.0` : divisione **reale**, e u non arriva mai a 1.
- `rand() / RAND_MAX` è una divisione **intera**: quasi sempre `0` !
- `RAND_MAX + 1` intero può andare in **overflow** (lezione 3).

Numeri interi in un intervallo

```
1   int k = a + rand() % (b - a + 1);   // in [a, b]
```

- `rand() % (b - a + 1)` è tra `0` e `b - a`; sommando `a` si ottiene `[a, b]`, estremi **inclusi**.
- Per esempio `rand() % 100 + 1` è tra 1 e 100.
- Alternativa che usa i bit "alti", in genere migliori: `a + (int) (u * (b - a + 1))`.
- Nei laboratori userete anche `drand48()` e `lrand48()`, inizializzate con `srand48(seme)`: `drand48()` dà direttamente un `double` in $[0, 1)$.

Il bias del resto

Supponiamo che il generatore dia gli interi da 0 a 9 e che vogliamo un valore in $[0, 2]$ con `r % 3`:

`r % 3`	valori di `r`	probabilità
0	0, 3, 6, 9	4/10
1	1, 4, 7	3/10
2	2, 5, 8	3/10

Se `RAND_MAX + 1` non è un multiplo dell'ampiezza, i valori **piccoli** sono favoriti. Con `RAND_MAX` $\approx 2 \times 10^9$ e intervalli piccoli l'effetto è trascurabile, ma esiste.

Verifica dell'uniformità

Un istogramma (lezione 18) di molti numeri u: ogni bin deve contenere circa $N/\texttt{NBIN}$ conteggi.

```c
#include <stdio.h>
#include <stdlib.h>

#define N    100000
#define NBIN 10

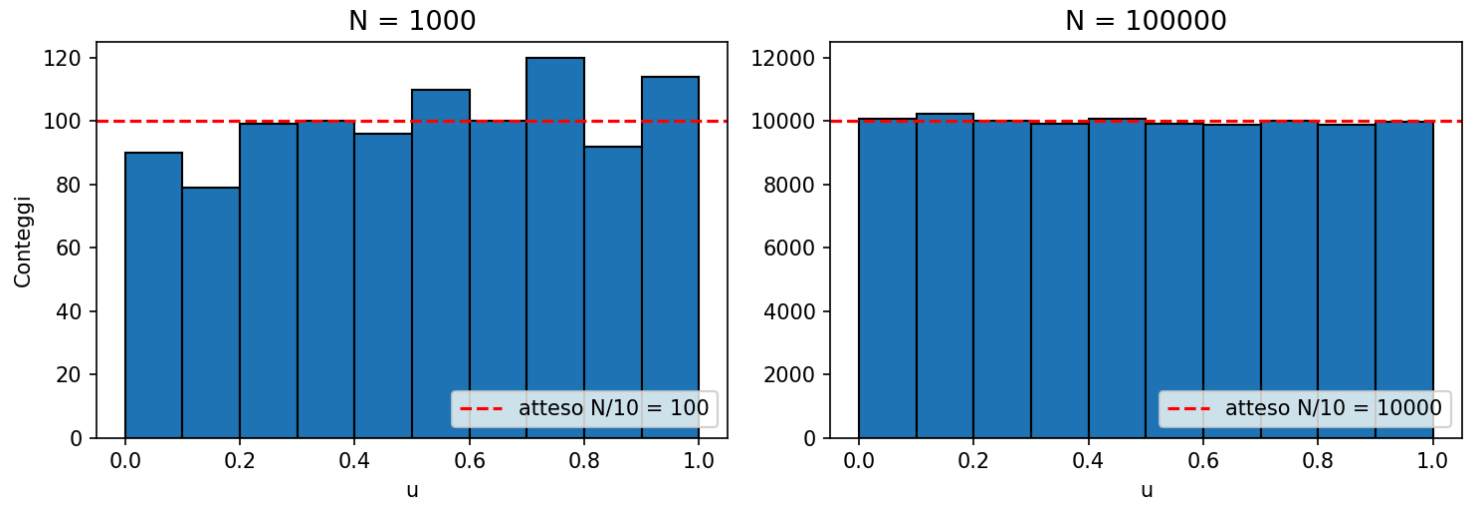
int main() {
    int conteggi[NBIN] = {0};
    int i, k;

    srand(12345);
    for (i = 0; i < N; i++) {
        double u = rand() / (RAND_MAX + 1.0);
        k = (int) (u * NBIN);          // xmin = 0, w = 1/NBIN
        conteggi[k]++;
    }
    for (k = 0; k < NBIN; k++) {
        printf("%.1f-%.1f  %d\n", k * 0.1, (k + 1) * 0.1,
               conteggi[k]);
    }
    return 0;
}
```

```
$ ./verifica
0.0-0.1  9974
0.1-0.2  10042
0.2-0.3  9987
0.3-0.4  10079
0.4-0.5  10001
0.5-0.6  9939
0.6-0.7  10003
0.7-0.8  10024
0.8-0.9  9988
0.9-1.0  9963
```

- Attesi 10000 per bin. Le differenze sono **fluttuazioni statistiche**, dell'ordine di $\sqrt{10000} = 100$.
- Poiché $u < 1$, `k` è sempre minore di `NBIN`: niente controllo dell'indice.

Uniformità e fluttuazioni



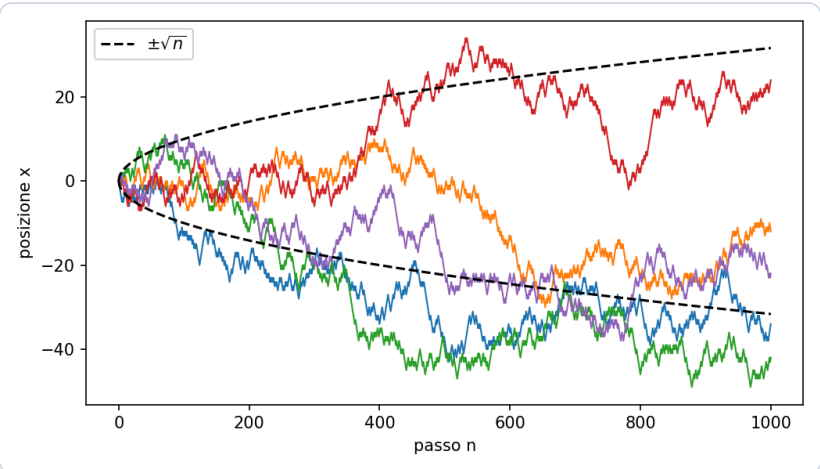
Con $N = 1000$: 100 conteggi attesi per bin, fluttuazioni di $\pm\sqrt{100} = \pm 10$, cioè il **10%**.

Con $N = 100000$: fluttuazioni di ± 100 su 10000, cioè l'**1%**.
L'errore relativo scala come $1/\sqrt{N}$.

Il cammino casuale

Una particella parte da $x = 0$ e a ogni passo si sposta di $+1$ o -1 con probabilità $1/2$ (come lanciando una moneta).

Cammino casuale e diffusione



5 cammini da 1000 passi ciascuno.

- Ogni cammino è diverso, ma la **distanza tipica** dall'origine cresce come $\sqrt{N_{passi}}$.
- Se i passi avvengono a intervalli regolari, $\langle x^2 \rangle \propto t$: è la legge della **diffusione**.
- Einstein (1905) la usò per spiegare il **moto browniano** e dimostrare l'esistenza degli atomi.
- Per fare un grafico come questo basta stampare `i` e `x` a ogni passo e usare `plt.plot` (lezione 9).

Decadimento radioattivo

N_0 nuclei; in un intervallo Δt ogni nucleo decade con probabilità $p = \lambda\,\Delta t$ (per Δt piccolo).

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define N0 10000
int main() {
    const double lambda = 0.05;      // costante di decadimento [1/s]
    const double dt = 0.1;           // passo temporale [s]
    const double p = lambda * dt;    // probabilità di decadere in dt
    int n = N0;                      // nuclei ancora presenti
    int passo, j;
    srand(2026);
    for (passo = 1; passo <= 600; passo++) {
        int decaduti = 0;
        for (j = 0; j < n; j++) {
            if (rand() / (RAND_MAX + 1.0) < p) {
                decaduti++;
            }
        }
        n -= decaduti;
        if (passo % 100 == 0) {
```

```
$ ./decadimento
   10   6043   6065.3
   20   3719   3678.8
   30   2201   2231.3
   40   1356   1353.4
   50    800    820.8
   60    467    497.9
```

- Colonne: t [s], nuclei simulati, $N_0 e^{-\lambda t}$.
- Per ogni nucleo si "estrae a sorte" se decade: `u < p` è vero con probabilità `p`.
- Il ciclo interno usa il valore **attuale** di `n`.
- Le differenze sono fluttuazioni, relativamente più grandi quando restano **pochi** nuclei.
- Compilate con `-lm` (per `exp`).